

SPM-Kit Data Hunter

Usage Guide v2.2.0

José Labarca Baeza

July 2026

Deep, resumable discovery of public AFM/SPM validation evidence

MIT License | <https://github.com/kegouro/spmkit-data-hunter>

Contents

1	Introduction	3
2	Installation	3
3	Core Concepts	4
3.1	Evidence Chains	4
3.2	Domain Relevance Gate	4
3.3	Utility Classes	4
3.4	Heuristic Levels: Gold, Silver, Bronze	5
3.5	Benchmark Score (0–100)	5
3.6	File Categories	5
4	Command Reference	5
4.1	Global Entry Point	5
4.2	<code>doctor</code> — Inspect Installation	6
4.3	<code>sources list</code> — List Source Adapters	6
5	Campaign Commands	6
5.1	<code>campaign create</code> — Define a Campaign	6
5.2	<code>campaign run</code> / <code>campaign resume</code>	6
5.3	<code>campaign status</code> / <code>campaign list</code>	6
5.4	<code>campaign pause</code> / <code>campaign stop</code>	6
5.5	<code>campaign verify</code> — Probe Remote Files	7
5.6	<code>campaign export</code> — Export Campaign Catalog	7
6	Download Commands	7
6.1	<code>download plan</code> — Preview Without Downloading	7
6.2	<code>download run</code> — Execute Downloads	7
7	Campaign Workflow — Step by Step	8
7.1	Campaign Lifecycle Statuses	9
8	Query Presets	9
9	Data Sources	9

10 Output Structure	10
11 Validation Philosophy	10
12 Development	10
13 Troubleshooting	11
14 Relationship to SPM-Kit	11

1 Introduction

Canonical documentation: This PDF is a printable companion. The Markdown version at `docs/usage-guide.md` is the authoritative reference and may contain additional detail. Both are verified against SPM-Kit Data Hunter v2.2.0.

SPM-Kit Data Hunter is a command-line tool that discovers, catalogs, classifies, and selectively downloads public AFM/SPM datasets from scientific repositories. It searches Zenodo, Figshare, and DataCite through official public APIs — not by scraping commercial websites.

Its purpose is to find and organize the evidence chains needed to validate scientific software for atomic force microscopy (AFM) and scanning probe microscopy (SPM). A validation chain requires more than isolated files: it needs native instrument data, processing methods or code, processed outputs, and publication context.

Data Hunter does **not** declare scientific ground truth. It provides structure, classification, and provenance so that human reviewers can make informed decisions.

Key Design Principles

- **Discovery is not validation** — a search result is a candidate, not a benchmark.
- **Raw-only datasets are reader fixtures**, not analysis validators.
- **Processed reference values are evidence**, not absolute truth.
- **Scientific review remains necessary** before any validation claim.

Note: Data Hunter is alpha software (v2.2.0). Expect improvements to classification heuristics, source coverage, and format intelligence in future releases.

2 Installation

Requirements: Python 3.11 or later, pip, git.

Clone and Set Up

```
git clone https://github.com/kegouro/spmkit-data-hunter.git
cd spmkit-data-hunter
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

On Windows PowerShell:

```
.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
```

Verify the Installation

```
spmkit-data-hunter doctor
spmkit-data-hunter sources list
pytest
ruff check .
```

The `doctor` command reports whether optional API tokens are detected (it never prints their values) and lists available source adapters.

Dependencies

Type	Packages
Runtime	<code>requests</code> (HTTP client), <code>tqdm</code> (progress bars)
Development	<code>pytest</code> (testing), <code>ruff</code> (linting/formatting)

Tip: Optional API tokens (`ZENODO_TOKEN`, `FIGSHARE_TOKEN`, `GITHUB_TOKEN`) may improve rate limits but are not required for public discovery.

3 Core Concepts

3.1 Evidence Chains

Data Hunter scores records by the completeness of their evidence chain:

`native data` → `method/code` → `processed output` → `publication`

Each link contributes to the benchmark score. A file alone tests a reader; a full chain enables validation.

3.2 Domain Relevance Gate

Before scoring, every record passes through a deterministic, offline domain relevance check. It detects AFM/SPM signals through:

- **Strong phrases:** “atomic force microscopy”, “scanning probe microscopy”, “KPFM”, etc.
- **Acronyms:** AFM, SPM, KPFM, MFM, EFM, STM.
- **Contextual families:** cantilever mechanics, topography, SPM software, force spectroscopy.
- **Native file extensions:** `.nid`, `.gwy`, `.jpk`, `.spm`, `.ibw`, `.sxm`, `.mdt`, `.sm4`, etc.

Records that fail the gate receive capped scores and cannot be Gold or Silver.

3.3 Utility Classes

Every record receives one of **seven utility classes**. This is the most important classification.

Class	Criteria	Valid Use
<code>benchmark_ready</code>	Distinct raw + processed assets + method/code	Analysis validation candidate (human review required)
<code>crosscheck_candidate</code>	Distinct raw + processed, incomplete method/code	Preliminary comparison, manual follow-up
<code>reader_fixture</code>	Raw/native data, no processed reference	Reader, parser, channel, robustness tests
<code>processed_ref_only</code>	Processed output, no recoverable raw input	Numerical context, format examples
<code>documentation_only</code>	Paper, script without usable data	Query expansion, provenance discovery
<code>incomplete</code>	Insufficient or ambiguous evidence	Manual triage, query refinement
<code>rejected</code>	Corrupt, empty, unsafe, irrelevant	None without resolving the issue

3.4 Heuristic Levels: Gold, Silver, Bronze

Gold, Silver, and Bronze are compact convenience labels that combine benchmark score with utility class. They are **NOT** declarations of scientific truth.

- **Gold** — `benchmark_ready` AND score ≥ 72 .
- **Silver** — `benchmark_ready` or `crosscheck_candidate` AND score ≥ 48 .
- **Bronze** — everything else.

3.5 Benchmark Score (0–100)

Signal	Points
Raw data detected	+32
Processed outputs detected	+24
Code or notebooks detected	+18
Documentation or methods detected	+10
Archive files present	+4
Text mentions raw/original data	+5
Text mentions results or processing	+5
Method or calibration signals	+8
Has DOI	+4
Links to related resources	+5
Recognizable open license	+4
No raw data detected	−18
Images/documents only, no data	−15
No public files	−30
Complete chain bonus (4+ categories)	+5

3.6 File Categories

Files are classified by extension with token-aware name analysis (word boundaries prevent substring traps like `drawings.csv` matching “raw”):

Category	Example Extensions
raw	<code>.nid .gwy .jpk .jpk-force .spm .ibw .sxm .mdt .sm4</code>
processed	<code>.csv .tsv .xlsx .json .tif .h5 .npy .npz .mat</code>
code	<code>.py .ipynb .m .r .jl .cpp .sh .toml</code>
documentation	<code>.md .pdf .tex .rst .docx .html</code>
image	<code>.png .jpg .svg .bmp .webp</code>
archive	<code>.zip .tar.gz .tar.bz2 .tar.xz .7z .rar</code>

4 Command Reference

The CLI has two modes: the **new subcommand tree** (campaign, download, doctor, sources) and the **legacy flag-only mode**. The new mode is recommended for all work.

4.1 Global Entry Point

```
spmkit-data-hunter [command]
--version      Print version and exit
--help         Print help and exit
```

4.2 doctor — Inspect Installation

```
spmkit-data-hunter doctor [--json]
```

Reports Python version, platform, user agent, detected API tokens (presence only, never values), and available source adapters.

4.3 sources list — List Source Adapters

```
spmkit-data-hunter sources list
```

Prints JSON with source name, role, file inventory support, checksum support, resume strategy, and auth requirements.

5 Campaign Commands

Campaigns persist progress in SQLite. They survive interruptions, can be paused/resumed/stopped, and export results to multiple formats.

5.1 campaign create — Define a Campaign

```
spmkit-data-hunter campaign create <name>
--output <path>           Output directory (default: ./spm_benchmarks)
--source <name>           Source adapter (repeatable)
--preset <name>           Query preset (repeatable)
--query <text>            Custom query (repeatable)
--page-size <n>           Records per page (default: 100)
--max-runtime <dur>       Time budget: 0=unlimited, 1h, 30m, 3600
--max-records <n>         Record budget: 0=unlimited
--heartbeat <n>           Heartbeat interval in seconds (default: 15)
--min-score <n>           Minimum score filter (default: 0)
--require-open-license    Only include records with open licenses
--rate-seconds <n>        Delay between API calls (default: 1.05)
--timeout <n>             HTTP timeout in seconds (default: 45.0)
```

The configuration is frozen at creation time.

5.2 campaign run / campaign resume

```
spmkit-data-hunter campaign run <name> [--output <path>]
spmkit-data-hunter campaign resume <name> [--output <path>]
```

5.3 campaign status / campaign list

```
spmkit-data-hunter campaign status <name> [--output <path>]
spmkit-data-hunter campaign list [--output <path>]
```

5.4 campaign pause / campaign stop

```
spmkit-data-hunter campaign pause <name>
spmkit-data-hunter campaign stop <name>
```

Requests graceful interruption at the next safe page checkpoint. **Ctrl+C** during a run is equivalent to pause.

5.5 campaign verify — Probe Remote Files

```
spmkit-data-hunter campaign verify <name>
--max-files <n>           Limit number of files to probe
--rate-seconds <n>       Delay between probes
--timeout <n>            HTTP timeout
```

Uses HEAD requests (falling back to one-byte range) to check reachability. Does **not** download file contents.

5.6 campaign export — Export Campaign Catalog

```
spmkit-data-hunter campaign export <name>
--output <path>          Campaign output directory
--target <path>          Export destination (defaults to --output)
```

Exports all records to JSON, JSONL, CSV, and Markdown files.

Warning: A campaign checkpoint advances only after the entire page is committed. A failed API partition is **never** marked exhausted — it retries on the next resume.

6 Download Commands

Discovery and download are deliberately separate. You discover first, then decide what to download.

6.1 download plan — Preview Without Downloading

```
spmkit-data-hunter download plan <campaign-name>
--output <path>          Campaign output directory
--level gold|silver|bronze Filter by level (repeatable)
--category <name>       Filter by file category (repeatable)
```

Reports record count, file count, known size in GiB, and count of files with unknown sizes. Nothing is downloaded.

6.2 download run — Execute Downloads

```
spmkit-data-hunter download run <campaign-name>
--output <path>          Campaign output directory
--level gold|silver|bronze Filter by level (repeatable)
--category <name>       Filter by category (repeatable)
--max-file-gb <n>       Per-file size limit (default: 2.0)
--max-record-gb <n>     Per-record total size limit (default: 10.0)
--accept-unbounded-downloads Allow unlimited sizes
--inspect-archives      List archive contents without extraction
```

```
--rate-seconds <n>      Delay between downloads (default: 1.05)
--timeout <n>           HTTP timeout (default: 120.0)
```

Downloads are resumable: already-downloaded files with matching checksums are skipped. Repository checksums (MD5) and local SHA-256 are recorded.

Warning: Unbounded downloads require `-accept-unbounded-downloads`. This is a deliberate safety acknowledgment, not a scientific filter.

7 Campaign Workflow — Step by Step

1. Create:

```
spmkit-data-hunter campaign create afm-scan \
  --preset all --source all \
  --max-runtime 2h --max-records 0 \
  --output spm_benchmarks
```

2. Run:

```
spmkit-data-hunter campaign run afm-scan --output spm_benchmarks
```

3. Check progress (from another terminal):

```
spmkit-data-hunter campaign status afm-scan
spmkit-data-hunter campaign list
```

4. Pause, resume, or stop:

```
spmkit-data-hunter campaign pause afm-scan
spmkit-data-hunter campaign resume afm-scan
spmkit-data-hunter campaign stop afm-scan
```

5. Verify remote file reachability:

```
spmkit-data-hunter campaign verify afm-scan
```

6. Export results:

```
spmkit-data-hunter campaign export afm-scan
```

7. Plan downloads:

```
spmkit-data-hunter download plan afm-scan \
  --level gold silver --category raw processed
```

8. Execute downloads:

```
spmkit-data-hunter download run afm-scan \
  --level gold silver \
  --category raw processed code \
  --max-file-gb 2 --max-record-gb 20 \
  --inspect-archives
```

7.1 Campaign Lifecycle Statuses

Status	Meaning
created	Configuration frozen, not yet executed
running	Actively searching and persisting
paused	Interrupted via Ctrl+C or pause request
stopped	Interrupted via stop request
completed	All partitions exhausted with no errors
completed_with_errors	All exhausted, some recoverable errors
failed	Unrecoverable error

Tip: A campaign can be resumed from **paused** or **stopped**. Checkpoints are page-granular. A resumed campaign skips exhausted partitions and retries failed ones.

8 Query Presets

Preset	Scope	Queries
all	Broad AFM/SPM discovery	8
topography	Height maps, profiles, roughness	4
force	Force curves, calibration, adhesion	4
kpfm	Surface potential, Kelvin probe	3
grains	Segmentation, particles, grain analysis	3
resonance	Cantilever thermal tune, resonance	3

Usage Examples

```
# Single preset
spmkit-data-hunter campaign create fc --preset force --source all

# Combine presets
spmkit-data-hunter campaign create multi --preset force --preset kpfm --
  source all

# Presets + custom queries
spmkit-data-hunter campaign create custom \
  --preset force \
  --query "single molecule force spectroscopy raw processed" \
  --source all
```

Custom queries are appended to preset queries and deduplicated. If no preset or custom query is provided, the all preset is used.

9 Data Sources

Source	API	Role	Files	Resume	Auth
Zenodo	api.zenodo.org	Direct repo	Yes	Page	Optional
Figshare	api.figshare.com/v2	Direct repo	Yes	Page	Optional
DataCite	api.datacite.org	Meta-index	Usually no	Cursor	Not req.

DataCite records are valuable for DOI discovery and publication linkage but typically lack file inventories — they are metadata evidence, not benchmark packages.

Planned adapters: OSF, Dataverse, Dryad, InvenioRDM, DSpace 7.

Each source adapter follows strict rules: cursors advance only after full page commit; failed partitions are never marked exhausted; replaying a page is safe (idempotent writes); source adapters fetch metadata only, never download dataset files directly.

10 Output Structure

```
spm_benchmarks/
  catalog.sqlite3      Normalized records, file inventories, scores
  campaigns.sqlite3    Campaign configs, checkpoints, events, stats
  catalog.json         JSON export
  catalog.jsonl        JSON Lines export
  catalog.csv          CSV export
  REPORT.md           Markdown report
  datasets/           Downloaded files organized by record
```

- `catalog.sqlite3` — source of truth for discovered records and file inventories.
- `campaigns.sqlite3` — source of truth for campaign state.
- Export files are views — regenerate anytime with `campaign export`.
- Both databases use WAL journal mode.

11 Validation Philosophy

This section is essential reading before using discovered data for scientific validation.

- **Discovery is not validation** — results are candidates, not benchmarks.
- **Raw-only files are reader fixtures** — a `.nid` file can test a parser, not an algorithm.
- **Processed values are references, not absolute truth** — units, calibration, and parameters matter.
- **Gwyddion is a reference, not an oracle** — version, parameters, and operation order must be recorded.

Data Hunter guarantees: structured discovery, classification, provenance linking, file reachability verification.

Data Hunter does NOT guarantee: scientific correctness certification, automated benchmark validation, ground truth declarations.

Read the full doctrine in `SCIENTIFIC_DATA_HUNTING_BIBLE.md`.

Before reusing a dataset: (1) read its license, (2) cite the version DOI, (3) do not redistribute restricted files, (4) preserve checksums and provenance, (5) record processing parameters, (6) consult an AFM/SPM practitioner.

12 Development

```
pip install -e ".[dev]"
pytest                      # 53 tests
ruff check .
ruff format --check .
spmkit-data-hunter --self-test
```

Project Structure

```
src/spmkit_data_hunter/
  __init__.py      Package exports
  __main__.py      python -m entry point
  cli.py           Unified command-line interface
  campaigns.py     Campaign model and SQLite store
  catalog_io.py    Record reconstruction from catalog
  engine.py        Campaign execution orchestration
  legacy.py        Models, classification, scoring, downloads
  sources.py       Paged source adapters
  verification.py  Lightweight remote file probes
  version.py       Version string
```

Every real false positive or API edge case should become a regression test.

13 Troubleshooting

Problem	Solution
Campaign won't start	Check it was created (<code>campaign list</code>), verify output directory, run <code>doctor</code> .
Unbounded downloads error	Pass <code>-accept-unbounded-downloads</code> to confirm.
Network errors during campaign	Engine catches exceptions per partition and continues. Retries on resume.
Campaign appears stuck	Check heartbeat (<code>campaign status</code>). Reduce <code>-rate-seconds</code> .
Corrupted downloads	Check checksums in catalog. Delete file and re-download.
SQLite database locked	Ensure one process per output directory. WAL recovery is automatic.

Inspecting campaign events directly:

```
sqlite3 spm_benchmarks/campaigns.sqlite3 \
"SELECT timestamp, level, event_type, message
FROM events ORDER BY id DESC LIMIT 20;"
```

14 Relationship to SPM-Kit

SPM-Kit Data Hunter is a companion project for **SPM-Kit** (<https://github.com/kegouro/spmkit>), an open-source AFM/SPM analysis toolkit.

- **SPM-Kit** performs AFM analysis: loading native formats, processing images, fitting force curves.
- **Data Hunter** discovers and curates public material for reader tests, cross-checks, and validation manifests.
- Data Hunter does **not** perform AFM analysis. SPM-Kit does **not** perform dataset discovery.

Together, they aim to make AFM/SPM software validation more reproducible, transparent, and data-driven.

Appendix A: Quick Reference Card

```
# Create and run
```

```

spmkit-data-hunter campaign create NAME --preset all --source all --max-
runtime 2h --max-records 0
spmkit-data-hunter campaign run NAME

# Manage
spmkit-data-hunter campaign status NAME
spmkit-data-hunter campaign pause NAME
spmkit-data-hunter campaign resume NAME
spmkit-data-hunter campaign stop NAME

# Verify and export
spmkit-data-hunter campaign verify NAME
spmkit-data-hunter campaign export NAME

# Download
spmkit-data-hunter download plan NAME --level gold silver
spmkit-data-hunter download run NAME --level gold --category raw processed

# Inspect
spmkit-data-hunter doctor
spmkit-data-hunter sources list

```

Appendix B: Environment Variables

Variable	Purpose	Required
GITHUB_TOKEN	GitHub API authentication	No
ZENODO_TOKEN	Zenodo API authentication	No
FIGSHARE_TOKEN	Figshare API authentication	No
OSF_TOKEN	OSF API (future)	No
DATAVERSE_TOKEN	Dataverse API (future)	No
DRYAD_TOKEN	Dryad API (future)	No

Appendix C: Verification Status Codes

Status	Meaning
not_checked	Default; no probe attempted
reachable	Remote object responded to HEAD or range request
rejected_url	URL is not an allowed public HTTPS URL
rejected_redirect	Redirect resolved to disallowed destination
empty	Remote object reports zero bytes
size_mismatch	Repository metadata size differs from remote size
failed	Request failed (timeout, DNS, connection refused)